Hardhat 3 + Ethers v6 最新部署完整教程

目录

Hardhat 3 新特性概述

主要改进

1. **Hardhat Ignition** - 全新的声明式部署系统
2. **更好的 TypeScript 支持** - 开箱即用的类型支持
3. **Viem 集成** - 除了 ethers.js，还支持 viem
4. **改进的插件系统** - 更模块化的架构
5. **性能优化** - 编译和测试速度提升
6. **EDR (Ethereum Development Runtime)** - 新的运行时引擎

版本要求

- Node.js >= 18.0.0
- npm >= 8.0.0 或 pnpm >= 8.0.0
- Hardhat >= 3.0.0
- Ethers >= 6.0.0

环境准备与配置

1. 全新项目初始化

```
# 创建项目目录
mkdir my-hardhat3-project
cd my-hardhat3-project

# 初始化 npm 项目
npm init -y

# 安装 Hardhat 3
npm install --save-dev hardhat

# 初始化 Hardhat 项目
npx hardhat init
```

初始化时会看到新的选项：

```
? What do you want to do? ...
> Create a JavaScript project
  Create a TypeScript project
  Create a TypeScript project (with Viem)
  Create an empty hardhat.config.js
  Quit
```

2. 安装核心依赖

使用 Ethers.js（推荐）

```
# Hardhat 3 + Ethers v6
npm install --save-dev @nomicfoundation/hardhat-toolbox

# 这会自动安装以下插件：
# - @nomicfoundation/hardhat-ethers
# - @nomicfoundation/hardhat-chai-matchers
# - @nomicfoundation/hardhat-verify
# - @nomicfoundation/hardhat-ignition-ethers
# - chai
# - ethers
# - hardhat-gas-reporter
# - solidity-coverage
```

使用 Viem（新选项）

```
# Hardhat 3 + Viem
npm install --save-dev @nomicfoundation/hardhat-toolbox-viem
```

3. 环境变量配置

创建 `.env` 文件：

```
# 私钥（不要提交到 Git！）
PRIVATE_KEY=your_private_key_without_0x_prefix

# RPC URLs
SEPOLIA_RPC_URL=https://eth-sepolia.g.alchemy.com/v2/YOUR_API_KEY
MAINNET_RPC_URL=https://eth-mainnet.g.alchemy.com/v2/YOUR_API_KEY

# Etherscan API
ETHERSCAN_API_KEY=your_etherscan_api_key

# Gas Reporter
COINMARKETCAP_API_KEY=your_coinmarketcap_api_key
REPORT_GAS=false
```

安装 dotenv：

```
npm install --save-dev dotenv
```

4. Hardhat 3 配置文件

创建 `hardhat.config.js`（JavaScript 版本）：

```
require("@nomicfoundation/hardhat-toolbox");
require("dotenv").config();

/** @type import('hardhat/config').HardhatUserConfig */
module.exports = {
  solidity: {
    version: "0.8.24", // Hardhat 3 推荐使用最新的 Solidity 版本
    settings: {
      optimizer: {
        enabled: true,
        runs: 200,
      },
      viaIR: false, // 可选：使用新的 IR 编译器
    },
  },

  networks: {
    // 本地网络
    hardhat: {
      chainId: 31337,
      // Hardhat 3 的新配置选项
      mining: {
        auto: true,
        interval: 0,
      },
    },

    localhost: {
```

```
      url: "http://127.0.0.1:8545",
      chainId: 31337,
    },

    // 测试网络
    sepolia: {
      url: process.env.SEPOLIA_RPC_URL || "",
      accounts: process.env.PRIVATE_KEY ? [process.env.PRIVATE_KEY] : [],
      chainId: 11155111,
    },

    // 主网
    mainnet: {
      url: process.env.MAINNET_RPC_URL || "",
      accounts: process.env.PRIVATE_KEY ? [process.env.PRIVATE_KEY] : [],
      chainId: 1,
    },
  },

  // Etherscan 验证配置
  etherscan: {
    apiKey: {
      sepolia: process.env.ETHERSCAN_API_KEY || "",
      mainnet: process.env.ETHERSCAN_API_KEY || "",
    },
  },

  // Gas Reporter 配置
  gasReporter: {
    enabled: process.env.REPORT_GAS === "true",
    currency: "USD",
    coinmarketcap: process.env.COINMARKETCAP_API_KEY,
    outputFile: "gas-report.txt",
    noColors: true,
  },

  // Hardhat 3 新增：路径配置
  paths: {
    sources: "./contracts",
    tests: "./test",
    cache: "./cache",
    artifacts: "./artifacts",
    ignition: "./ignition", // Ignition 模块目录
  },
};
```

或 TypeScript 版本 `hardhat.config.ts`：

```
import { HardhatUserConfig } from "hardhat/config";
import "@nomicfoundation/hardhat-toolbox";
import "dotenv/config";
```

```
const config: HardhatUserConfig = {
  solidity: {
    version: "0.8.24",
    settings: {
      optimizer: {
        enabled: true,
        runs: 200,
      },
    },
  },

  networks: {
    hardhat: {
      chainId: 31337,
    },
    sepolia: {
      url: process.env.SEPOLIA_RPC_URL || "",
      accounts: process.env.PRIVATE_KEY ? [process.env.PRIVATE_KEY] : [],
      chainId: 11155111,
    },
    mainnet: {
      url: process.env.MAINNET_RPC_URL || "",
      accounts: process.env.PRIVATE_KEY ? [process.env.PRIVATE_KEY] : [],
      chainId: 1,
    },
  },

  etherscan: {
    apiKey: process.env.ETHERSCAN_API_KEY,
  },

  gasReporter: {
    enabled: process.env.REPORT_GAS === "true",
    currency: "USD",
    coinmarketcap: process.env.COINMARKETCAP_API_KEY,
  },
};

export default config;
```

Hardhat 2 vs Hardhat 3 主要变化

配置变化

特性	Hardhat 2	Hardhat 3
Node 版本	>= 14	>= 18
默认 Solidity	0.8.19	0.8.24
TypeScript 配置	需手动配置	开箱即用
部署工具	脚本/hardhat-deploy	Ignition（新增）
插件安装	单独安装	toolbox 统一安装

新增的 Ignition 部署系统

Hardhat 3 引入了全新的 **Hardhat Ignition** 部署系统：

特点：

- 声明式部署定义
- 自动依赖管理
- 支持部署回滚和恢复
- 内置部署状态管理
- 跨链部署支持

传统脚本 vs Ignition：

```
// 传统脚本方式
async function main() {
  const Token = await ethers.getContractFactory("Token");
  const token = await Token.deploy("My Token", "MTK");
  await token.waitForDeployment();
  console.log("Token:", token.target);
}

// Ignition 模块方式
const TokenModule = buildModule("TokenModule", (m) => {
  const token = m.contract("Token", ["My Token", "MTK"]);
  return { token };
});
```

工具链更新

```
# Hardhat 2
npm install --save-dev @nomiclabs/hardhat-ethers ethers

# Hardhat 3
npm install --save-dev @nomicfoundation/hardhat-toolbox
```

完整项目初始化

项目结构

```
my-hardhat3-project/
├── contracts/              # 智能合约
│   └── MyToken.sol
├── ignition/               # Ignition 部署模块（新）
│   └── modules/
│       └── TokenModule.js
├── scripts/                # 传统部署脚本
│   └── deploy.js
├── test/                   # 测试文件
│   └── MyToken.test.js
├── deployments/            # 部署记录（Ignition 自动生成）
├── .env                    # 环境变量
├── .gitignore
├── hardhat.config.js       # Hardhat 配置
├── package.json
└── README.md
```

.gitignore

```
node_modules
.env
coverage
coverage.json
typechain
typechain-types

# Hardhat files
cache
artifacts
deployments/*/solcInputs

# Hardhat Ignition default folder for deployments
ignition/deployments
```

智能合约开发

示例合约

创建 `contracts/MyToken.sol`：

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

/**
 * @title MyToken
 * @dev 简单的 ERC20-like Token 合约示例
```

```
 */
contract MyToken {
    string public name;
    string public symbol;
    uint8 public decimals;
    uint256 public totalSupply;

    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;

    event Transfer(address indexed from, address indexed to, uint256 value);
    event Approval(address indexed owner, address indexed spender, uint256 value);

    constructor(string memory _name, string memory _symbol, uint256 _initialSupply) {
        name = _name;
        symbol = _symbol;
        decimals = 18;
        totalSupply = _initialSupply * 10**uint256(decimals);
        balanceOf[msg.sender] = totalSupply;
        emit Transfer(address(0), msg.sender, totalSupply);
    }

    function transfer(address to, uint256 value) public returns (bool) {
        require(balanceOf[msg.sender] >= value, "Insufficient balance");
        require(to != address(0), "Invalid address");

        balanceOf[msg.sender] -= value;
        balanceOf[to] += value;

        emit Transfer(msg.sender, to, value);
        return true;
    }

    function approve(address spender, uint256 value) public returns (bool) {
        allowance[msg.sender][spender] = value;
        emit Approval(msg.sender, spender, value);
        return true;
    }

    function transferFrom(address from, address to, uint256 value) public returns (bool) {
        require(balanceOf[from] >= value, "Insufficient balance");
        require(allowance[from][msg.sender] >= value, "Allowance exceeded");
        require(to != address(0), "Invalid address");

        balanceOf[from] -= value;
        balanceOf[to] += value;
        allowance[from][msg.sender] -= value;

        emit Transfer(from, to, value);
        return true;
    }
}
```

编译合约

```
# 编译所有合约
npx hardhat compile

# 清理并重新编译
npx hardhat clean
npx hardhat compile

# 查看合约大小
npx hardhat size-contracts
```

脚本部署方式

基础部署脚本

创建 `scripts/deploy.js`：

```
const hre = require("hardhat");

async function main() {
  console.log("=".repeat(50));
  console.log("开始部署 MyToken 合约");
  console.log("=".repeat(50));

  // 获取网络信息
  const network = await hre.ethers.provider.getNetwork();
  console.log("\n网络信息:");
  console.log("- 网络名称:", hre.network.name);
  console.log("- Chain ID:", network.chainId);

  // 获取部署者信息
  const [deployer] = await hre.ethers.getSigners();
  console.log("\n部署者信息:");
  console.log("- 地址:", deployer.address);

  const balance = await hre.ethers.provider.getBalance(deployer.address);
  console.log("- 余额:", hre.ethers.formatEther(balance), "ETH");

  // 部署参数
  const TOKEN_NAME = "My Token";
  const TOKEN_SYMBOL = "MTK";
  const INITIAL_SUPPLY = 1_000_000; // 100万

  console.log("\n部署参数:");
  console.log("- 名称:", TOKEN_NAME);
  console.log("- 符号:", TOKEN_SYMBOL);
  console.log("- 初始供应量:", INITIAL_SUPPLY.toLocaleString());
```

```
// 部署合约
console.log("\n正在部署合约...");
const MyToken = await hre.ethers.getContractFactory("MyToken");
const myToken = await MyToken.deploy(TOKEN_NAME, TOKEN_SYMBOL, INITIAL_SUPPLY);

// 等待部署完成
await myToken.waitForDeployment();

const contractAddress = await myToken.getAddress();

console.log("\n✓ 部署成功! ");
console.log("- 合约地址:", contractAddress);

// 获取部署交易信息
const deployTx = myToken.deploymentTransaction;
if (deployTx) {
  console.log("- 部署交易:", deployTx.hash);
  console.log("- 区块号:", deployTx.blockNumber);
}

// 验证合约状态
console.log("\n验证合约状态:");
const tokenName = await myToken.name();
const tokenSymbol = await myToken.symbol();
const supply = await myToken.totalSupply();

console.log("- 名称:", tokenName);
console.log("- 符号:", tokenSymbol);
console.log("- 总供应量:", hre.ethers.formatUnits(supply, 18));

// 保存部署信息
const deployment = {
  network: hre.network.name,
  chainId: Number(network.chainId),
  contractAddress: contractAddress,
  deployer: deployer.address,
  timestamp: new Date().toISOString(),
  constructorArgs: [TOKEN_NAME, TOKEN_SYMBOL, INITIAL_SUPPLY],
};

// 写入文件
const fs = require("fs");
const path = require("path");
const deploymentsDir = path.join(__dirname, "../deployments");

if (!fs.existsSync(deploymentsDir)) {
  fs.mkdirSync(deploymentsDir, { recursive: true });
}

fs.writeFileSync(
  path.join(deploymentsDir, `MyToken-${hre.network.name}.json`),
  JSON.stringify(deployment, null, 2)
```

```
  );

  console.log("\n✓ 部署信息已保存到 deployments/");

  // Etherscan 验证
  if (hre.network.name !== "hardhat" && hre.network.name !== "localhost") {
    console.log("\n等待区块确认后验证合约...");
    await myToken.deploymentTransaction.wait(5);

    try {
      await hre.run("verify:verify", {
        address: contractAddress,
        constructorArguments: [TOKEN_NAME, TOKEN_SYMBOL, INITIAL_SUPPLY],
      });
      console.log("✓ 合约验证成功! ");
    } catch (error) {
      console.log("✗ 合约验证失败:", error.message);
    }
  }

  console.log("\n" + "=".repeat(50));
  console.log("部署完成! ");
  console.log("=".repeat(50) + "\n");

  return myToken;
}

main()
  .then(() => process.exit(0))
  .catch((error) => {
    console.error(error);
    process.exit(1);
  });
```

运行部署脚本

```
# 部署到本地 Hardhat 网络
npx hardhat run scripts/deploy.js

# 部署到本地节点
npx hardhat node  # 终端 1
npx hardhat run scripts/deploy.js --network localhost  # 终端 2

# 部署到 Sepolia 测试网
npx hardhat run scripts/deploy.js --network sepolia

# 部署到主网
npx hardhat run scripts/deploy.js --network mainnet
```

Ignition 部署模块（Hardhat 3 新增）

什么是 Hardhat Ignition?

Ignition 是 Hardhat 3 引入的全新部署系统，提供：

- **声明式部署** - 定义"是什么"而非"怎么做"
- **状态管理** - 自动跟踪部署状态
- **错误恢复** - 支持中断后继续部署
- **依赖管理** - 自动处理合约依赖关系
- **可重复性** - 确保部署的一致性

创建 Ignition 模块

创建 `ignition/modules/TokenModule.js`：

```
const { buildModule } = require("@nomicfoundation/hardhat-ignition/modules");

/**
 * Ignition 模块：部署 MyToken 合约
 */
const TokenModule = buildModule("TokenModule", (m) => {
  // 定义参数（可以在部署时覆盖）
  const tokenName = m.getParameter("name", "My Token");
  const tokenSymbol = m.getParameter("symbol", "MTK");
  const initialSupply = m.getParameter("initialSupply", 1_000_000);

  // 部署合约
  const token = m.contract("MyToken", [tokenName, tokenSymbol, initialSupply]);

  // 返回部署的合约
  return { token };
});

module.exports = TokenModule;
```

高级 Ignition 模块（多合约部署）

创建 `ignition/modules/TokenEcosystemModule.js`：

```
const { buildModule } = require("@nomicfoundation/hardhat-ignition/modules");

const TokenEcosystemModule = buildModule("TokenEcosystemModule", (m) => {
  // 1. 部署 Token 合约
  const token = m.contract("MyToken", ["Ecosystem Token", "ECO", 10_000_000]);

  // 2. 部署依赖 Token 的合约（例如 TokenSale）
  const salePrice = m.getParameter("salePrice", "1000000000000000"); // 0.001 ETH
  const tokenSale = m.contract("TokenSale", [token, salePrice]);
```

```
  // 3. 调用合约方法进行初始化
  // 给 TokenSale 合约转账一些 Token
  const transferAmount = 1_000_000n * 10n**18n; // 100万 tokens
  m.call(token, "transfer", [tokenSale, transferAmount], {
    id: "transfer_to_sale",
  });

  // 4. 设置角色或权限（如果有的话）
  // m.call(token, "grantRole", [MINTER_ROLE, tokenSale]);

  // 返回所有部署的合约
  return { token, tokenSale };
});

module.exports = TokenEcosystemModule;
```

使用参数配置

创建 `ignition/parameters/sepolia.json`：

```
{
  "TokenModule": {
    "name": "My Token (Sepolia)",
    "symbol": "MTK",
    "initialSupply": 1000000
  }
}
```

创建 `ignition/parameters/mainnet.json`：

```
{
  "TokenModule": {
    "name": "My Production Token",
    "symbol": "MPT",
    "initialSupply": 10000000
  }
}
```

运行 Ignition 部署

```
# 部署到本地网络（使用默认参数）
npx hardhat ignition deploy ignition/modules/TokenModule.js

# 部署到 Sepolia（使用自定义参数）
npx hardhat ignition deploy ignition/modules/TokenModule.js \
  --network sepolia \
  --parameters ignition/parameters/sepolia.json

# 部署到主网
```

```
npx hardhat ignition deploy ignition/modules/TokenModule.js \
  --network mainnet \
  --parameters ignition/parameters/mainnet.json

# 验证已部署的合约
npx hardhat ignition verify chain-11155111

# 查看部署状态
npx hardhat ignition status chain-11155111

# 可视化部署计划（不实际部署）
npx hardhat ignition visualize ignition/modules/TokenModule.js
```

Ignition 部署后的状态管理

部署后，Ignition 会在 `ignition/deployments/` 目录下创建部署记录：

```
ignition/
└── deployments/
    └── chain-11155111/  # Sepolia
        ├── deployed_addresses.json
        ├── journal.jsonl
        └── artifacts/
```

`deployed_addresses.json` 示例：

```
{
  "TokenModule#MyToken": "0x5FbDB2315678afecb367f032d93F642f64180aa3"
}
```

在代码中使用已部署的合约

```
const hre = require("hardhat");
const fs = require("fs");
const path = require("path");

async function main() {
  // 读取 Ignition 部署的地址
  const chainId = (await hre.ethers.provider.getNetwork()).chainId;
  const deploymentPath = path.join(
    __dirname,
    `../ignition/deployments/chain-${chainId}/deployed_addresses.json`
  );

  const addresses = JSON.parse(fs.readFileSync(deploymentPath, "utf8"));
  const tokenAddress = addresses["TokenModule#MyToken"];

  // 获取合约实例
  const token = await hre.ethers.getContractAt("MyToken", tokenAddress);
```

```
  // 与合约交互
  const name = await token.name();
  console.log("Token Name:", name);
}

main();
```

单元测试

基础测试（Ethers v6 + Chai）

创建 `test/MyToken.test.js`：

```
const { expect } = require("chai");
const { ethers } = require("hardhat");
const { loadFixture } = require("@nomicfoundation/hardhat-toolbox/network-helpers");

describe("MyToken 合约测试", function () {
  // Fixture: 部署合约并返回所需的对象
  async function deployTokenFixture() {
    const [owner, addr1, addr2] = await ethers.getSigners();

    const MyToken = await ethers.getContractFactory("MyToken");
    const token = await MyToken.deploy("My Token", "MTK", 1_000_000);
    await token.waitForDeployment();

    return { token, owner, addr1, addr2 };
  }

  describe("部署测试", function () {
    it("应该正确设置 token 名称和符号", async function () {
      const { token } = await loadFixture(deployTokenFixture);

      expect(await token.name()).to.equal("My Token");
      expect(await token.symbol()).to.equal("MTK");
      expect(await token.decimals()).to.equal(18);
    });

    it("应该给部署者分配全部初始供应量", async function () {
      const { token, owner } = await loadFixture(deployTokenFixture);

      const totalSupply = await token.totalSupply();
      const ownerBalance = await token.balanceOf(owner.address);

      expect(ownerBalance).to.equal(totalSupply);
      expect(ownerBalance).to.equal(ethers.parseUnits("1000000", 18));
    });
  });
```

```
describe("转账功能", function () {
  it("应该成功转账 tokens", async function () {
    const { token, owner, addr1 } = await loadFixture(deployTokenFixture);

    const amount = ethers.parseUnits("100", 18);

    await expect(token.transfer(addr1.address, amount))
      .to.emit(token, "Transfer")
      .withArgs(owner.address, addr1.address, amount);

    expect(await token.balanceOf(addr1.address)).to.equal(amount);
  });

  it("应该在余额不足时失败", async function () {
    const { token, addr1, addr2 } = await loadFixture(deployTokenFixture);

    await expect(
      token.connect(addr1).transfer(addr2.address, ethers.parseUnits("1", 18))
    ).to.be.revertedWith("Insufficient balance");
  });

  it("应该正确更新余额", async function () {
    const { token, owner, addr1, addr2 } = await loadFixture(deployTokenFixture);

    const initialOwnerBalance = await token.balanceOf(owner.address);

    // 转账给 addr1
    await token.transfer(addr1.address, ethers.parseUnits("100", 18));

    // 转账给 addr2
    await token.transfer(addr2.address, ethers.parseUnits("50", 18));

    const finalOwnerBalance = await token.balanceOf(owner.address);

    expect(finalOwnerBalance).to.equal(
      initialOwnerBalance - ethers.parseUnits("150", 18)
    );
  });
});

describe("授权功能", function () {
  it("应该批准并转移 tokens", async function () {
    const { token, owner, addr1, addr2 } = await loadFixture(deployTokenFixture);

    const amount = ethers.parseUnits("100", 18);

    // 批准 addr1 花费 owner 的 tokens
    await token.approve(addr1.address, amount);
    expect(await token.allowance(owner.address, addr1.address)).to.equal(amount);

    // addr1 从 owner 转账到 addr2
    await token.connect(addr1).transferFrom(owner.address, addr2.address, amount);
```

```
      expect(await token.balanceOf(addr2.address)).to.equal(amount);
      expect(await token.allowance(owner.address, addr1.address)).to.equal(0);
    });

    it("应该在超出授权额度时失败", async function () {
      const { token, owner, addr1, addr2 } = await loadFixture(deployTokenFixture);

      await token.approve(addr1.address, ethers.parseUnits("50", 18));

      await expect(
        token.connect(addr1).transferFrom(
          owner.address,
          addr2.address,
          ethers.parseUnits("100", 18)
        )
      ).to.be.revertedWith("Allowance exceeded");
    });
  });

  describe("Gas 优化测试", function () {
    it("应该报告转账的 gas 消耗", async function () {
      const { token, addr1 } = await loadFixture(deployTokenFixture);

      const tx = await token.transfer(addr1.address, ethers.parseUnits("100", 18));
      const receipt = await tx.wait();

      console.log("      Gas 消耗:", receipt.gasUsed.toString());

      // 断言 gas 消耗在合理范围内
      expect(receipt.gasUsed).to.be.lt(100_000n);
    });
  });
});
```

运行测试

```
# 运行所有测试
npx hardhat test

# 运行特定测试文件
npx hardhat test test/MyToken.test.js

# 使用 grep 过滤测试
npx hardhat test --grep "转账"

# 显示 gas 报告
REPORT_GAS=true npx hardhat test

# 显示测试覆盖率
npx hardhat coverage
```

```
# 并行测试（更快）
npx hardhat test --parallel
```

高级测试：时间操作和快照

```
const { time } = require("@nomicfoundation/hardhat-toolbox/network-helpers");

describe("时间相关测试", function () {
  it("应该在锁定期后允许提取", async function () {
    const { contract } = await loadFixture(deployFixture);

    // 快进时间 1 天
    await time.increase(24 * 60 * 60);

    // 快进到特定时间戳
    const futureTime = Math.floor(Date.now() / 1000) + 3600;
    await time.increaseTo(futureTime);

    // 获取最新区块时间
    const latestTime = await time.latest();
    console.log("当前区块时间:", latestTime);
  });
});
```

TypeScript 支持

TypeScript 项目配置

Hardhat 3 对 TypeScript 有更好的支持。

安装 TypeScript 依赖

```
npm install --save-dev typescript ts-node @types/node @types/mocha @types/chai
```

TypeScript 配置文件

创建 `tsconfig.json`：

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "commonjs",
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "strict": true,
    "skipLibCheck": true,
    "resolveJsonModule": true,
```

```
    "outDir": "dist",
    "declaration": true
  },
  "include": ["./scripts", "./test", "./ignition", "./hardhat.config.ts"],
  "files": ["./hardhat.config.ts"]
}
```

TypeScript 部署脚本

创建 `scripts/deploy.ts`：

```
import { ethers } from "hardhat";

async function main() {
  const [deployer] = await ethers.getSigners();

  console.log("部署者地址:", deployer.address);
  console.log(
    "部署者余额:",
    ethers.formatEther(await ethers.provider.getBalance(deployer.address))
  );

  const MyToken = await ethers.getContractFactory("MyToken");
  const token = await MyToken.deploy("My Token", "MTK", 1_000_000);

  await token.waitForDeployment();

  const address = await token.getAddress();
  console.log("MyToken 部署到:", address);

  // 类型安全的合约调用
  const name: string = await token.name();
  const symbol: string = await token.symbol();
  const totalSupply: bigint = await token.totalSupply();

  console.log("名称:", name);
  console.log("符号:", symbol);
  console.log("总供应量:", ethers.formatUnits(totalSupply, 18));
}

main()
  .then(() => process.exit(0))
  .catch((error) => {
    console.error(error);
    process.exit(1);
  });
```

TypeScript 测试文件

创建 `test/MyToken.test.ts`：

```
import { expect } from "chai";
import { ethers } from "hardhat";
import { loadFixture } from "@nomicfoundation/hardhat-toolbox/network-helpers";
import { MyToken } from "../typechain-types";
import { SignerWithAddress } from "@nomicfoundation/hardhat-ethers/signers";

describe("MyToken", function () {
  async function deployTokenFixture() {
    const [owner, addr1, addr2]: SignerWithAddress[] = await ethers.getSigners();

    const MyTokenFactory = await ethers.getContractFactory("MyToken");
    const token: MyToken = await MyTokenFactory.deploy("My Token", "MTK", 1_000_000);
    await token.waitForDeployment();

    return { token, owner, addr1, addr2 };
  }

  describe("部署", function () {
    it("应该正确设置名称", async function () {
      const { token } = await loadFixture(deployTokenFixture);
      expect(await token.name()).to.equal("My Token");
    });
  });

  describe("转账", function () {
    it("应该成功转账", async function () {
      const { token, owner, addr1 } = await loadFixture(deployTokenFixture);

      const amount = ethers.parseUnits("100", 18);
      await expect(token.transfer(addr1.address, amount))
        .to.emit(token, "Transfer")
        .withArgs(owner.address, addr1.address, amount);
    });
  });
});
```

运行 TypeScript 脚本：

```
npx hardhat run scripts/deploy.ts --network sepolia
npx hardhat test
```

Viem 集成（可选）

Hardhat 3 支持使用 Viem 替代 Ethers.js。

安装 Viem

```
npm install --save-dev @nomicfoundation/hardhat-toolbox-viem
```

配置 Viem

`hardhat.config.js`：

```
require("@nomicfoundation/hardhat-toolbox-viem");

module.exports = {
  solidity: "0.8.24",
  // 其他配置...
};
```

使用 Viem 部署

创建 `scripts/deploy-viem.js`：

```
const hre = require("hardhat");

async function main() {
  const [deployer] = await hre.viem.getWalletClients();

  console.log("部署者:", deployer.account.address);

  const myToken = await hre.viem.deployContract("MyToken", [
    "My Token",
    "MTK",
    1_000_000n,
  ]);

  console.log("MyToken 部署到:", myToken.address);

  // 调用合约
  const name = await myToken.read.name();
  console.log("Token 名称:", name);
}

main();
```

Viem 测试

```
const { expect } = require("chai");
const hre = require("hardhat");
const { loadFixture } = require("@nomicfoundation/hardhat-toolbox-viem/network-helpers");

describe("MyToken (Viem)", function () {
  async function deployTokenFixture() {
    const [owner, addr1] = await hre.viem.getWalletClients();

    const token = await hre.viem.deployContract("MyToken", [
      "My Token",
```

```
      "MTK",
      1_000_000n,
    ]);

    const publicClient = await hre.viem.getPublicClient();

    return { token, owner, addr1, publicClient };
  }

  it("应该部署成功", async function () {
    const { token } = await loadFixture(deployTokenFixture);

    const name = await token.read.name();
    expect(name).to.equal("My Token");
  });

  it("应该转账成功", async function () {
    const { token, addr1 } = await loadFixture(deployTokenFixture);

    const hash = await token.write.transfer([addr1.account.address, 1000n]);
    expect(hash).to.be.a("string");
  });
});
```

常见问题与解决方案

1. Hardhat 3 迁移问题

问题: 从 Hardhat 2 升级到 3 时出错

解决方案:

```
# 1. 清理旧的缓存和构建文件
npx hardhat clean
rm -rf node_modules package-lock.json

# 2. 更新 package.json
npm install --save-dev hardhat@^3.0.0
npm install --save-dev @nomicfoundation/hardhat-toolbox

# 3. 更新配置文件
# 将 @nomiclabs/* 改为 @nomicfoundation/*

# 4. 重新安装依赖
npm install

# 5. 重新编译
npx hardhat compile
```

2. Ignition 部署失败

问题: `Ignition module not found`

解决方案:

```
# 确保安装了 Ignition
npm install --save-dev @nomicfoundation/hardhat-ignition-ethers

# 确保目录结构正确
mkdir -p ignition/modules

# 检查模块导出
# module.exports = TokenModule; ✓
# export default TokenModule; ✗ (JavaScript 中不要用)
```

3. TypeScript 类型错误

问题: `Cannot find module 'typechain-types'`

解决方案:

```
# 编译合约以生成类型
npx hardhat compile

# TypeScript 会自动生成 typechain-types/
# 在 tsconfig.json 中包含:
{
  "include": ["./typechain-types"]
}
```

4. Gas 估算错误

问题: `Transaction reverted without a reason`

解决方案:

```
// 添加详细的 revert 原因
try {
  await contract.someFunction();
} catch (error) {
  console.log("完整错误:", error);

  // 使用 callStatic 预执行
  try {
    await contract.someFunction.staticCall();
  } catch (staticError) {
    console.log("静态调用错误:", staticError);
  }
}
```

5. Hardhat 网络连接问题

问题: `Error: could not detect network`

解决方案:

```
// hardhat.config.js
module.exports = {
  networks: {
    sepolia: {
      url: process.env.SEPOLIA_RPC_URL,
      accounts: [process.env.PRIVATE_KEY],
      timeout: 60000, // 增加超时时间
      httpHeaders: {  // 可选：添加请求头
        "User-Agent": "hardhat",
      },
    },
  },
};
```

最佳实践

1. 项目结构最佳实践

```
project/
├── contracts/
│   ├── core/               # 核心合约
│   ├── interfaces/         # 接口定义
│   ├── libraries/          # 库合约
│   └── mocks/             # 测试用的模拟合约
├── ignition/
│   ├── modules/           # 部署模块
│   └── parameters/        # 部署参数
├── scripts/               # 工具脚本
├── test/
│   ├── unit/              # 单元测试
│   └── integration/       # 集成测试
└── docs/                  # 文档
```

2. Git 工作流

`.gitignore`:

```
node_modules
.env
.env.local

# Hardhat
```

```
cache
artifacts
typechain
typechain-types

# Ignition
ignition/deployments/chain-*/
!ignition/deployments/chain-*/deployed_addresses.json

# Coverage
coverage
coverage.json

# Build
dist
build
```

3. 环境变量管理

创建多个环境文件：

```
.env                 # 本地开发
.env.sepolia         # Sepolia 测试网
.env.mainnet         # 主网（不提交！）
```

使用脚本加载：

```
// scripts/deploy.js
require("dotenv").config({
  path: `.env.${process.env.NETWORK || "local"}`
});
```

运行：

```
NETWORK=sepolia npx hardhat run scripts/deploy.js --network sepolia
```

4. 安全检查清单

```
async function securityChecklist() {
  // 1. 检查网络
  const chainId = (await ethers.provider.getNetwork()).chainId;
  if (chainId === 1n) {
    console.log("警告：即将部署到主网！");
    // 添加额外确认步骤
  }

  // 2. 检查余额
  const balance = await ethers.provider.getBalance(deployer.address);
  if (balance < ethers.parseEther("0.1")) {
```

```
    throw new Error("余额不足");
  }

  // 3. 估算 Gas
  const gasEstimate = await contract.transfer.estimateGas(to, amount);
  console.log("预估 Gas:", gasEstimate);

  // 4. 模拟执行
  await contract.transfer.staticCall(to, amount);
  console.log("✓ 静态调用成功");

  // 5. 实际执行
  const tx = await contract.transfer(to, amount);
  await tx.wait();
}
```

5. CI/CD 集成

`.github/workflows/test.yml`:

```yaml
name: Test

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'

      - name: Install dependencies
        run: npm ci

      - name: Compile contracts
        run: npx hardhat compile

      - name: Run tests
        run: npx hardhat test

      - name: Coverage
        run: npx hardhat coverage
```

6. 性能优化

```
// hardhat.config.js
module.exports = {
  solidity: {
    version: "0.8.24",
    settings: {
      optimizer: {
        enabled: true,
        runs: 200,  // 根据使用频率调整
      },
      viaIR: true,  // 启用 IR 优化（实验性）
    },
  },

  // 并行编译
  parallel: true,

  // 缓存策略
  cache: {
    enabled: true,
  },
};
```

总结

Hardhat 3 核心优势

1. **Ignition 部署系统** - 声明式、可恢复、状态管理
2. **更好的 TypeScript 支持** - 开箱即用
3. **Viem 集成** - 现代化的 Web3 库选择
4. **性能提升** - 更快的编译和测试
5. **统一工具链** - hardhat-toolbox 简化配置

推荐工作流

```
# 1. 初始化项目
npx hardhat init

# 2. 开发合约
# contracts/MyToken.sol

# 3. 编写测试
# test/MyToken.test.js
npx hardhat test

# 4. 创建 Ignition 模块
# ignition/modules/TokenModule.js

# 5. 本地测试部署
npx hardhat ignition deploy ignition/modules/TokenModule.js
```

```
# 6. 部署到测试网
npx hardhat ignition deploy ignition/modules/TokenModule.js \
  --network sepolia \
  --parameters ignition/parameters/sepolia.json

# 7. 验证合约
npx hardhat ignition verify chain-11155111

# 8. 部署到主网
npx hardhat ignition deploy ignition/modules/TokenModule.js \
  --network mainnet \
  --parameters ignition/parameters/mainnet.json
```

快速命令参考

```
# 编译
npx hardhat compile
npx hardhat clean

# 测试
npx hardhat test
npx hardhat test --parallel
npx hardhat coverage

# 部署（Ignition）
npx hardhat ignition deploy <module>
npx hardhat ignition verify <chain-id>
npx hardhat ignition status <chain-id>

# 部署（传统脚本）
npx hardhat run scripts/deploy.js --network <network>

# 验证
npx hardhat verify --network <network> <address> <args>

# 网络
npx hardhat node
npx hardhat console --network <network>
```

参考资源

- **Hardhat 3 官方文档**: https://hardhat.org/
- **Hardhat Ignition**: https://hardhat.org/ignition/docs/getting-started
- **Ethers v6 文档**: https://docs.ethers.org/v6/
- **Viem 文档**: https://viem.sh/
- **Hardhat Toolbox**: https://hardhat.org/hardhat-runner/plugins/nomicfoundation-hardhat-toolbox
- **示例项目**: https://github.com/NomicFoundation/hardhat-boilerplate